

4 Rubik’s Cube Challenge

A programmer would like to use an OOP approach to store information regarding the contestants of a Rubik’s cube challenge.

For class `Contestant`, the data that will be stored include:

- `name` (string)
- `time` (float)

For class `beginner`, the additional data stored include:

- `practice_hours` (integer)

For class `professional`, the additional data stored include:

- `competitions_won` (integer)

In class `Contestant`, the following class methods are to be implemented.

Method	Description
<code>Contestant(name, time)</code>	Constructor of the class <code>Contestant</code>
<code>get_time()</code>	Getter for attribute <code>time</code>
<code>set_time(new_time)</code>	Setter for attribute <code>time</code>
<code>__str__()</code>	The string representation of a <code>Contestant</code> instance. The string returned starts with a "C-", followed by the first 8 characters of the name in uppercase excluding spaces. If the number of characters in the name is fewer than 8, fill up the remaining spaces with "_". Finally, append the <code>time</code> with at most 2 decimal places with a bracket and a "s". For example, <pre>>>> print(str(Contestant("Kan Min Coh", 35.32))) C-KANMINCO(35.32s) >>> print(str(Contestant("Man Kin", 15.1))) C-MANKIN__(15.1s)</pre>

[RVHS/2024/Q4 Cont']

In class `Beginner`, the following class methods are to be implemented.

Method	Description
<code>Beginner(name, time, practice_hours)</code>	Constructor of the class <code>Beginner</code>
<code>__str__()</code>	This is a polymorphed function. The string representation of a <code>Beginner</code> instance. Instead of starting with "C-", the string returned starts with a "B-". For example, <pre>>>> print(str(Beginner("Kan Min Coh", 35.32))) B-KANMINCO(35.32s) >>> print(str(Beginner("Man Kin", 15.1))) B-MANKIN__(15.1s)</pre>

In class `Professional`, the following class methods are to be implemented.

Method	Description
<code>Professional(name, time, competitions_won)</code>	Constructor of the class <code>Professional</code>
<code>__str__()</code>	This is a polymorphed function. The string representation of a <code>Professional</code> instance. Instead of starting with "C-", the string returned starts with a "P-". For example, <pre>>>> print(str(Professional("Kan Min Coh", 35.32))) P-KANMINCO(35.32s) >>> print(str(Professional("Man Kin", 15.1))) P-MANKIN__(15.1s)</pre>

[RVHS/2024/Q4 Cont']

Task 4.1

Implement the 3 classes as mentioned above.

[6]

Test your function using the following call:

- `print(str(Contestant("ChanChan Chan", 35.01)))`
- `print(str(Beginner("Li uLi uLi u", 55.54, 300)))`
- `print(str(Professional("OhOhOh", 6.34, 60)))`

[1]

Task 4.2

Write a function `readfile(filename)` that:

- takes a string value, `filename` which contains data stored in either one of the following formats:
 - `Beginner,<name>,<time>,<practice hour>`
 - `Professional,<name>,<time>,<competition won>`
- create an appropriate instance for each line in the file
- add all instances created in a list and return it

[3]

Task 4.3

Implement the class `Node` and class `BST` to store the instances created in Task 4.2. The description of the two classes are as follows.

In class `Node`, the following attributes are to be implemented.

[1]

Attribute	Description
<code>player</code>	This is to store the <code>Beginner</code> or <code>Professional</code> instance.
<code>left</code>	This points to the left sub-tree which is another <code>Node</code> instance
<code>right</code>	This points to the right sub-tree which is another <code>Node</code> instance
Method	Description
<code>Node(new_player)</code>	Constructor of the class <code>Node</code> where <code>new_player</code> is either a <code>Beginner</code> or <code>Professional</code> instance. <code>new_player</code> is assigned as the attribute <code>player</code> in the <code>Node</code> instance.

[RVHS/2024/Q4 Cont']

In class `BST`, the following attribute and methods are to be implemented.

Attribute	Description
<code>root</code>	This is to store the root of the BST which is a <code>Node</code> instance.
Method	Description
<code>BST()</code>	Constructor of the class <code>BST</code>
<code>Insert(player)</code>	Insert <code>player</code> which is either a <code>Beginner</code> or <code>Professional</code> instance into the BST based on its <code>time</code> attribute. Lower time should be inserted to the left side of the tree.
<code>inorder()</code>	Display the string representation of the attribute <code>player</code> of all the nodes in the BST in order.
<code>count_professtional()</code>	This recursive function counts and returns the total number of professionals in the BST.
<code>find_best_beginner()</code>	This recursive function finds the beginner with the lowest time in the BST. The <code>Beginner</code> instance should be returned.

[12]

Test your function using the following:

```
bst = BST()
players = readfile("rubik.txt")
for player in players:
    bst.insert(player)
bst.inorder()
print(bst.count_professional())
print(bst.find_best_beginner())
```

[2]